Practical No 02

A) Write the program with Function Overloading.

Project Name: FunctionOverloading

Project Type: Console app

Program Code:

I. Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FunctionOverloading
{
    class Calculator
    {
        public int Add(int a, int b)
        {
            Console.WriteLine($"Add(int, int)called");
            return a + b;
        }

        public int Add(int a, int b, int c)
        {
            Console.WriteLine($"Add(int,int,int)called");
            return a + b + c;
        }

        public double Add(double a, double b)
        {
            Console.WriteLine($"Add(double,double)called");
            return a + b;
        }

        public string Add(string a, string b)
        {
            Console.WriteLine($"Add(string,string)called");
            return a + b;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();
            Console.WriteLine("=== Function Overloading Demo ===");
            Console.WriteLine(calc.Add(10, 20)); //30
            Console.WriteLine(calc.Add(10, 20, 30)); //60
            Console.WriteLine(calc.Add(1.5, 2.5)); //4.0

            Console.WriteLine(calc.Add("Hello, ", "World!"));
            Console.ReadKey();
        }
    }
}
```

B) Write the program with Inheritance (all types)

Project Name: Floydtriangle

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace InheritanceDemo
{
    class Animal
    {
        public void Eat() => Console.WriteLine("Animal is eating");
    }

    class Dog : Animal
    {
        public void Bark() => Console.WriteLine("Dog is barking");
    }

    class GrandParent
    {
        public void Method1() => Console.WriteLine("GrandParent method");
    }

    class Parent : GrandParent
    {
        public void Method2() => Console.WriteLine("Parent method");
    }

    class Child : Parent
    {
        public void Method3() => Console.WriteLine("Child method");
    }

    class Shape
    {
        public void Draw() => Console.WriteLine("Drawing a shape");
    }

    class Circle : Shape
    {
        public void Info() => Console.WriteLine("This is a circle");
    }

        class Rectangle : Shape
    {
        public void Info() => Console.WriteLine("This is a rectangle");
    }

    interface IFlyable
    {
        void Fly();
    }

    interface ISwimable
```

```
    {
        void Swim();
    }

    class Duck : IFlyable, ISwimable
    {
        public void Fly() => Console.WriteLine("Duck can fly");
        public void Swim() => Console.WriteLine("Duck can swim");
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("---Single Inheritance---");
            Dog d = new Dog();
            d.Eat(); d.Bark();

            Console.WriteLine("---Multilevel Inheritance---");
            Child c = new Child();
            c.Method1(); c.Method2(); c.Method3();

            Console.WriteLine("---Hierarchical Inheritance---");
            Circle cir = new Circle(); cir.Draw(); cir.Info();
            Rectangle rec = new Rectangle(); rec.Draw(); rec.Info();

            Console.WriteLine("---Multiple Inheritance(interface)---");
            Duck duck = new Duck();
            duck.Fly(); duck.Swim();
            Console.ReadKey();
        }
    }
}
```

C) Write the program with Constructor Overloading

Project Name: Constructor

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Constructor
{
    class Student
    {
        public Student()
        {
            Console.WriteLine("Default Constructor Called");
        }

        public Student(string name)
        {
            Console.WriteLine("Student Name:" + name);
        }

        public Student(string name, int age)
        {
            Console.WriteLine("Student Name:" + name);
            Console.WriteLine("Student Age:" + age);
        }
    }

    class Program
    {
        static void Main()
        {
            Student s1 = new Student();
            Student s2 = new Student("Shub");
            Student s3 = new Student("Raj",19);
            Console.ReadLine();
        }
    }
}
```

D) Write the program with Interfaces

Project Name: Interface

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Interface
{
    interface IShape
    {
        void Area();
    }

    class Square:IShape
    {
        int side = 5;
        public void Area()
        {
            Console.WriteLine("Area of Square =" + (side * side));
        }
    }

    class Program
    {
        static void Main()
        {
            Square s = new Square();
            s.Area();
            Console.ReadLine();
        }
    }
}
```

E) Write the program with Using Delegates and events

Project Name: Delegatesvents

Project Type: Console App

Program Code:

I.Filename: Program.cs

```
using System;
using System.Text;
using System.Threading.Tasks;

namespace DelegatesEvents
{

    public delegate void NotifyHandler(string message);

    class EventPublisher
    {
        public event NotifyHandler OnDataProcessed;

        public void ProcessData(string data)
        {
            Console.WriteLine($"Processing data: {data}");
            OnDataProcessed?.Invoke($"Data '{data}' has been processed!");
        }
    }

    class EventSubscriber
    {
        public void HandleEvent(string message)
        {
            Console.WriteLine($"[Subscriber] Received: {message}");
        }
    }


    delegate int MathOperation(int x, int y);

    class Program
    {
        static int Multiply(int a, int b) => a * b;
        static int Add(int a, int b) => a + b;

        static void Main(string[] args)
        {
            Console.WriteLine("=== Delegates Demo ===");

            MathOperation op = Multiply;
            Console.WriteLine($"Multiply(4,5) = {op(4, 5)}");

            op = Add;
            Console.WriteLine($"Add(4,4) = {op(4, 4)}");

            Console.WriteLine("\n=== Event Demo ===");

            EventPublisher publisher = new EventPublisher();
            EventSubscriber subscriber = new EventSubscriber();

            publisher.OnDataProcessed += subscriber.HandleEvent;

            publisher.ProcessData("Student Record");
```

```
            publisher.ProcessData("Salary Data");

            Console.ReadLine();
        }
    }
}
```

F) Write the program with Exception handling

Project Name: ExceptionHandling

Project Type: Console App

Program Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ExceptionHandling
{
    class Program
    {
        static void Main()
        {
            int a, b, result;

            try
            {
                Console.WriteLine("Enter Frist Number: ");
                a = Convert.ToInt32(Console.ReadLine());
                Console.WriteLine("Enter Second Number: ");
                b = Convert.ToInt32(Console.ReadLine());

                result = a / b;
                Console.WriteLine("Result=" + result);
            }

            catch(DivideByZeroException)
            {
                Console.WriteLine("Cannot divide by zero");
            }

            catch(FormatException)
            {
                Console.WriteLine("Invaild Input");
            }

            finally
            {
                Console.WriteLine("Program Ended");
            }
            Console.ReadLine();
        }
    }
}
```